

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий  
Кафедра прикладной математики

Отчет защищен с оценкой \_\_\_\_\_

Преподаватель \_\_\_\_\_  
(подпись)

«\_\_\_» \_\_\_\_\_ 2026 г.

Отчет  
По лабораторной работе №6

«Файлы»

по дисциплине «Программирование – 2 семестр»

Студент группы ПИ-54 Дворников М. С.

Преподаватель доцент, к.т.н. Егорова Е.В.

Барнаул 2026

Задание 1 (№7). Написать программу создания, вывода и обработки файла.

N7.

Дан файл F (создать его), компонентами которого являются целые числа, не равные 0. Числа в файле записаны в следующем порядке: ++ -- ++ -- ..., то есть два положительных числа, два отрицательных числа и т.д. Число компонентов файла F делится на 12.

Переписать компоненты файла F в файл H так, чтобы в файле H числа шли в следующем порядке: +++ --- +++ ..., то есть три положительных числа, три отрицательных числа и т.д. Допустимо использование вспомогательного файла.

Текст программы 1:

```
#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
```

```
#define DATA "data.txt"
```

```
#define NDATA "ndata.txt"
```

```
// Создание файла
```

```
void make_file();
```

```
// Чтение данных с файла
```

```
void read_data(FILE* fpntr);
```

```
// Преобразование данных файла
```

```
FILE* change_data(FILE* fpntr);
```

```
int main() {
```

```
    SETIO;
```

```
    system("color F0");
```

```
    do {
```

```
        system("cls");
```

```
        printf("Доброе утро ;)\n");
```

```
        FILE* fpntr = fopen(DATA, "r");
```

```
        while (fpntr == NULL) {
```

```
            printf("\nФайл с данными не найден. Запускаю ввод файла...\n\n");
```

```
            make_file();
```

```
            fpntr = fopen("data.txt", "r");
```

```
        }
```

```
        printf("\nДанные получены:\n");
```

```
        read_data(fpntr);
```

```
        printf("\nНажмите любую кнопку, чтобы начать преобразование файла\n");
```

```
        _getch();
```

```
        printf("\n\nНачинаю преобразование файла...\n");
```

```
        fpntr = change_data(fpntr);
```

```
        printf("Файл преобразован. Полученные данные:\n");
```

```
        read_data(fpntr);
```

```
        fclose(fpntr);
```

```
        printf("\nВыполнение программы закончено. Чтобы выйти нажмите ESC.\n");
```

```
        printf("Чтобы выполнить программу ещё раз, нажмите любую другую кнопку");
```

```
    } while (_getch() != 27);
```

```
}
```

```
// Создание файла
```

```
void make_file() {
```

```
    FILE* fpntr = fopen(DATA, "w");
```

```
    printf("Введите количество чисел (кратно 12), которые вы хотите ввести: ");
```

```
    int n;
```

```
    do scanf("%d", &n);
```

```
    while (n < 12 || n % 12);
```

```
    printf("Введите числа в формате \"++ -- ++ -- ++ --\":\n");
```

```
    int num;
```

```
    for (int i = 0; i < n - 1; i++) {
```

```

        scanf("%d", &num);
        fprintf(fpntr, "%d ", num);
    }
    scanf("%d", &num);
    fprintf(fpntr, "%d", num);
    fclose(fpntr);
}

// Чтение данных с файла
void read_data(FILE* fpntr) {
    rewind(fpntr);
    int num;
    while (fscanf(fpntr, "%d", &num) != EOF)
        printf("%d ", num);
}

// Преобразование данных файла
FILE* change_data(FILE* fpntr) {
    FILE* new_fpntr = fopen(NDATA, "w");
    int numbers[12];
    rewind(fpntr);
    while (fscanf(fpntr, "%d", numbers) != EOF) {
        for (int i = 1; i < 12; i++) fscanf(fpntr, "%d", numbers + i);
        fprintf(new_fpntr, "%d %d %d ", numbers[0], numbers[1], numbers[4]);
        fprintf(new_fpntr, "%d %d %d ", numbers[2], numbers[3], numbers[6]);
        fprintf(new_fpntr, "%d %d %d ", numbers[5], numbers[8], numbers[9]);
        fprintf(new_fpntr, "%d %d %d ", numbers[7], numbers[10], numbers[11]);
    }
    fclose(fpntr);
    fclose(new_fpntr);
    new_fpntr = fopen(NDATA, "r");
    return new_fpntr;
}

```

Тесты программы 1:

Файла нет:

Доброе утро ;)

Файл с данными не найден. Запускаю ввод файла...

Введите количество чисел (кратно 12), которые вы хотите ввести: 36

Введите числа в формате "++ -- ++ -- ++ --":

1 2 -3 -4 5 6 -7 -8 9 10 -11 -12 1 2 -3 -4 5 6 -7 -8 9 10 -11 -12  
13 14 -15 -16 17 18 -19 -20 21 22 -23 -24

Данные получены:

1 2 -3 -4 5 6 -7 -8 9 10 -11 -12 1 2 -3 -4 5 6 -7 -8 9 10 -11 -12 13 14 -15 -16 17 18 -19 -20 21 22 -23 -24

Нажмите любую кнопку, чтобы начать преобразование файла

Начинаю преобразование файла...

Файл преобразован. Полученные данные:

1 2 5 -3 -4 -7 6 9 10 -8 -11 -12 1 2 5 -3 -4 -7 6 9 10 -8 -11 -12 13 14 17 -15 -16 -19 18 21 22 -20 -23 -24

Выполнение программы закончено. Чтобы выйти нажмите ESC.

Чтобы выполнить программу ещё раз, нажмите любую другую кнопку

Файл есть:

Доброе утро ;)

Данные получены:

1 2 -3 -4 5 6 -7 -8 9 10 -11 -12

Нажмите любую кнопку, чтобы начать преобразование файла

Начинаю преобразование файла...

Файл преобразован. Полученные данные:

1 2 5 -3 -4 -7 6 9 10 -8 -11 -12

Выполнение программы закончено. Чтобы выйти нажмите ESC.

Чтобы выполнить программу ещё раз, нажмите любую другую кнопку

Создание файлов работает:

 data.txt

 ndata.txt

## Задание 2 (№10). Написать программу создания, вывода и обработки файла. N10.

Дан символьный файл (создать его), содержащий слова. Слова в тексте разделяются пробелами и знаками препинания.

Получить 2 наиболее часто встречающихся слова и число их повторений. Если таких слов более 2, выдать соответствующую информацию для каждого из них.

Текст программы 2:

```
#define SETIO SetConsoleCP(1251); SetConsoleOutputCP(1251); // Ввод и вывод на русском
#define UTF SetConsoleCP(65001); SetConsoleOutputCP(65001); // Ввод и вывод на кодировке UTF

#define DATA "data.txt"
#define SEPS " ,!?:;"
#define WORD_LEN 20
#define WORDS 200

// Создание файла
void make_file();

// Чтение данных с файла
void read_data(FILE* fpntr);

// Найти 2 наиболее часто встречающихся слова
int find_words(FILE* fpntr);

int main() {
    SETIO;
    system("color F0");
    do {
        system("cls");
        printf("Доброе утро ;)\n");
        FILE* fpntr = fopen(DATA, "r");
        while (fpntr == NULL) {
            printf("\nФайл с данными не найден. Запускаю ввод файла...\n\n");
            make_file();
            fpntr = fopen("data.txt", "r");
        }
        printf("\nДанные получены:\n");
        read_data(fpntr);
        printf("\nНажмите любую кнопку, чтобы продолжить:\n");
        _getch();
        if (find_words(fpntr) == -1) printf("Файл пуст!\n");
        fclose(fpntr);
        printf("\nВыполнение программы закончено. Чтобы выйти нажмите ESC.\n");
        printf("Чтобы выполнить программу ещё раз, нажмите любую другую кнопку");
    } while (_getch() != 27);
}

// Создание файла
void make_file() {
    FILE* fpntr = fopen(DATA, "w");
    printf("Введите ваш текст. Чтобы закончить, нажмите Enter\n");
    char symb;
    while ((symb = getc(stdin)) != '\n') {
        UTF;
        putc(symb, fpntr);
        SETIO;
    }
    fclose(fpntr);
    SETIO;
}
```

```

// Чтение данных с файла
void read_data(FILE* fpntr) {
    char symb;
    rewind(fpntr);
    while ((symb = getc(fpntr)) != EOF) putc(symb, stdout);
}

// Найти 2 наиболее часто встречающихся слова
int find_words(FILE* fpntr) {
    int flag = 1; // Программа выполнила свою работу корректно
    // Формируем массив слов
    rewind(fpntr);
    char array[WORDS][WORD_LEN + 1]; // Массив слов
    int last_l = 0; // Индекс последней буквы
    int last_w = 0; // Индекс последнего слова
    char symb; // Символьный буфер
    while ((symb = getc(fpntr)) != EOF) {
        if (!strchr(SEPS, symb)) {
            if (last_l == WORD_LEN) { // Если буква уже не поместится
                // Разделим слово на 2
                flag = 0;
                array[last_w++][last_l] = '\0';
                last_l = 0;
            }
            if (last_w == WORDS) { // Если слово уже не поместится
                flag = 0;
                break;
            }
            array[last_w][last_l++] = symb;
        }
        else if (last_l > 0) {
            array[last_w++][last_l++] = '\0';
            last_l = 0;
        }
    }
    if (last_w < WORDS && last_l != 0) { // Если слово не успело завершиться
        array[last_w++][last_l] = '\0';
    }
    if (last_w == 0) return -1;
    // Найдём, сколько раз какое слово встречается
    int counter1 = 0, counter2 = 0; // Максимальная встречаемость
    int counters[WORDS]; // Массив, который хранит встречаемость всех слов в массиве
    for (int w = 0; w < last_w; w++) {
        int cur_counter = 0; // Сколько раз встречается это слово
        for (int i = 0; i < last_w; i++)
            if (!_strcmp(array[w], array[i])) cur_counter += 1;
        counters[w] = cur_counter;
        if (cur_counter > counter1) {
            counter2 = counter1;
            counter1 = cur_counter;
        }
        else if (cur_counter > counter2 && cur_counter != counter1)
            counter2 = cur_counter;
    }
    // Отсортируем слова по убыванию их встречаемости
    for (int k = 1; k < last_w; k++)
        for (int i = 0; i < last_w - k; i++)
            if (counters[i] < counters[i + 1]) {
                // (counters[i] == counters[i + 1] && _strcmp(array[i], array[i +
1]))) {
                    // Меняем местами встречаемость
                    int buffer = counters[i];
                    counters[i] = counters[i + 1];
                    counters[i + 1] = buffer;
                    // Меняем местами слова в массиве
                    for (int j = 0; j < WORD_LEN + 1; j++) {

```

```

        char buffer = array[i][j];
        array[i][j] = array[i + 1][j];
        array[i + 1][j] = buffer;
    }
}
// Отсортируем слова в алфавитном порядке
for (int k = 1; k < last_w; k++)
    for (int i = 0; i < last_w - k; i++)
        if (counters[i] == counters[i + 1]
            && _strcmp(array[i], array[i + 1]) > 0) {
            // Меняем местами слова в массиве
            for (int j = 0; j < WORD_LEN + 1; j++) {
                char buffer = array[i][j];
                array[i][j] = array[i + 1][j];
                array[i + 1][j] = buffer;
            }
        }
// Удалим повторяющиеся слова
int shift = 0; // Сдвиг массива
for (int i = 0; i < last_w - shift; i++) {
    strcpy(array[i], array[i + shift]);
    counters[i] = counters[i + shift];
    shift += counters[i] - 1;
}
if (!flag) printf("\nВНИМАНИЕ: Возможна потеря данных.");
printf("\nСамые часто встречающиеся слова:\n");
int n; // Сколько слов вывели
// Слова с максимальной встречаемостью
for (n = 0; n < last_w - shift && counters[n] == counter1; n++)
    printf("%s: %d\n", array[n], counters[n]);
// Слова с почти максимальной встречаемостью, если первых мало
if (n == 1)
    for (; n < last_w - shift && counters[n] == counter2; n++)
        printf("%s: %d\n", array[n], counters[n]);
return flag;
}

```

Тесты программы 2:

Файла нет:

Доброе утро ;)

Файл с данными не найден. Запускаю ввод файла...

Введите ваш текст. Чтобы закончить, нажмите Enter  
апельсин, кошка, сок: апельсин. кошка! дырка

Данные получены:

апельсин, кошка, сок: апельсин. кошка! дырка

Нажмите любую кнопку, чтобы продолжить:

Самые часто встречающиеся слова:

апельсин: 2

кошка: 2

Выполнение программы закончено. Чтобы выйти нажмите ESC.

Чтобы выполнить программу ещё раз, нажмите любую другую кнопку

Файл создан:

10\_task.exe

data.txt

Если файл есть:

 data.txt – Блокнот

Файл Правка Формат Вид Справка

раз два раз два раз три

Доброе утро ;)

Данные получены:

раз два раз два раз три

Нажмите любую кнопку, чтобы продолжить:

Самые часто встречающиеся слова:

раз: 3

два: 2

Выполнение программы закончено. Чтобы выйти нажмите ESC.

Чтобы выполнить программу ещё раз, нажмите любую другую кнопку